

# LAB 1B - Part2

## Simulación e Implementación de Circuitos con Arduino

La presente evaluación será recibida sólo si se ha cumplido con los checkpoints de todas las preguntas en el **LABORATORIO** con el docente del curso.

La entrega debe constar de:

1. **Introducción** (incluir párrafo introductorio):
  1. **Marco Teórico** (definiciones **con referencias**)
  2. **Estado del Arte** (trabajos de investigación, artículos, tesis, páginas **con referencias**).
2. **Metodología** (explicación de componentes e implementos utilizados y diagramas de flujo/bloques).
3. **Desarrollo de la experiencia** (incluye discusión)
4. **Conclusiones** (una por cada experiencia realizada **como mínimo**)
5. **Referencias** (formato IEEE)

### Consideraciones:

- Subir únicamente el PDF del informe a Canvas, usar un link para el repositorio, incluir fotos de paso a paso de cada implementación y un vídeo mostrando el funcionamiento de las experiencias. Sin embargo, para simulaciones, considere la placa Arduino UNO disponible en Tinkercad.
- Considere una placa de Arduino MEGA2560 tal cual existe en el laboratorio.
- Solo un miembro del grupo sube el informe.
- No olvidar colocar los nombres de todos los integrantes.

## Contenido

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>4. Implementación en Protoboard.....</b>                                        | <b>4</b>  |
| <b>5. Control de Tiempos y Eventos Asíncronos.....</b>                             | <b>6</b>  |
| 5.1. Conexión.....                                                                 | 6         |
| 5.2. Instrucciones.....                                                            | 6         |
| Se pide.....                                                                       | 7         |
| <b>6. Sistema de Acceso Matricial y Feedback Serial.....</b>                       | <b>8</b>  |
| 6.1. Conexión.....                                                                 | 8         |
| 6.2. Instrucciones.....                                                            | 8         |
| <b>7. Checkpoint Opcional: Visualización Dinámica en Matriz 8x8 (MAX7219).....</b> | <b>10</b> |
| 7.1. Conexión.....                                                                 | 10        |
| 7.2. Instrucciones.....                                                            | 10        |
| <b>ANEXO A. Códigos de Colores de las Resistencias.....</b>                        | <b>11</b> |
| <b>ANEXO B. Uso del Protoboard.....</b>                                            | <b>11</b> |
| 6.1. Conexión básico.....                                                          | 12        |
| 6.1.1. Conexión en serie.....                                                      | 12        |
| 6.1.2. Conexión en paralelo.....                                                   | 13        |
| <b>ANEXO C. Alimentación de protoboard con Arduino.....</b>                        | <b>13</b> |
| <b>ANEXO D. Potenciómetro.....</b>                                                 | <b>14</b> |
| 8.1. Configuración como resistencia variable (reóstato).....                       | 14        |
| 8.2. Configuración como divisor de tensión.....                                    | 14        |
| <b>ANEXO E. El Diodo LED (Light Emitting Diode).....</b>                           | <b>15</b> |
| <b>ANEXO F. Instrumentos y Formas de Medición.....</b>                             | <b>16</b> |
| 10.1. Medición de Intensidad de Corriente ("I").....                               | 16        |
| 10.2. Medición de Tensión o Voltaje ("V" o "U").....                               | 16        |
| <b>ANEXO G. Programación con Arduino: Interfaz de Arduino.....</b>                 | <b>17</b> |
| <b>ANEXO H. Botón con Arduino.....</b>                                             | <b>18</b> |
| <b>ANEXO I. Monitor Serial.....</b>                                                | <b>19</b> |
| <b>ANEXO J. Teclado Matricial.....</b>                                             | <b>22</b> |
| <b>ANEXO K. Matriz de Puntos 8x8 con Controlador MAX7219.....</b>                  | <b>24</b> |

## Parte II. Arduino

### Resumen de Funciones Esenciales de Arduino

Esta sección sirve como hoja de referencia rápida (Cheat Sheet) para la sintaxis y comandos más utilizados en la programación de microcontroladores basados en el framework de Arduino.

**1. Entradas y Salidas Digitales (GPIO)** Estas funciones gestionan el comportamiento discreto (0V o 5V) de los pines de la placa.

- **pinMode(pin, modo):** Configura el pin especificado para que se comporte como entrada o salida.
  - Parámetros: pin (número del pin), modo (INPUT, OUTPUT, o INPUT\_PULLUP).
  - Ejemplo: pinMode(13, OUTPUT);
- **digitalWrite(pin, valor):** Escribe un valor lógico alto (5V) o bajo (0V) en un pin digital previamente configurado como salida.
  - Parámetros: pin, valor (HIGH o LOW).
  - Ejemplo: digitalWrite(13, HIGH);
- **digitalRead(pin):** Lee el estado de un pin digital. Retorna si el pin está recibiendo voltaje (HIGH) o si está a tierra (LOW).
  - Retorno: HIGH o LOW.
  - Ejemplo: int estadoBoton = digitalRead(2);

**2. Entradas y Salidas Analógicas** Utilizadas para leer sensores continuos o simular salidas analógicas mediante modulación por ancho de pulsos (PWM).

- **analogRead(pin):** Lee el valor de un pin analógico (A0 a A5). El Arduino posee un conversor Analógico a Digital (ADC) de 10 bits, lo que mapea los voltajes de entrada entre 0 y 5V a valores enteros.
  - Retorno: Un entero entre 0 y 1023.
  - Ejemplo: int valorPotenciometro = analogRead(A0);
- **analogWrite(pin, valor):** Escribe un valor analógico (onda PWM) en un pin digital compatible (pines marcados con ~). Se usa para variar el brillo de un LED o la velocidad de un motor.
  - Parámetros: pin, valor (un entero entre 0 y 255).
  - Ejemplo: analogWrite(9, 127); // 50% de ciclo de trabajo (brillo medio)

**3. Gestión del Tiempo y Retardos** Funciones críticas para controlar la temporización y frecuencia de ejecución del código.

- **delay(ms):** Pausa la ejecución del programa durante la cantidad de tiempo especificada. Es una función bloqueante (el microcontrolador no puede hacer nada más mientras dure la pausa).
  - Parámetros: ms (milisegundos).
  - Ejemplo: delay(1000); // Pausa de 1 segundo
- **millis():** Retorna la cantidad de milisegundos transcurridos desde que la placa comenzó a ejecutar el programa actual. Es vital para implementar multitarea y temporizadores no bloqueantes.
  - Retorno: Un número entero largo sin signo (unsigned long).
  - Ejemplo: unsigned long tiempoActual = millis();

- **delayMicroseconds(us):** Pausa el programa por una cantidad específica de microsegundos (útil para protocolos de alta velocidad o sensores ultrasónicos).

**4. Comunicación Serial (Monitor Serial)** Permite el intercambio de mensajes de texto entre la placa y la computadora.

- **Serial.begin(baudrate):** Inicializa el hardware de comunicación serial y establece la velocidad en baudios.
  - Parámetros: baudrate (típicamente 9600 o 115200).
- **Serial.print(dato) / Serial.println(dato):** Envía datos al monitor serial. La variante println añade un salto de línea (Enter) al final.
- **Serial.available():** Obtiene el número de bytes (caracteres) disponibles para leer en el puerto serie.
  - Retorno: Un número entero indicando los bytes en el buffer.
- **Serial.read():** Lee el primer byte entrante del buffer serie.
  - Retorno: El primer carácter disponible (tipo char o int).

#### 5. Funciones Matemáticas de Utilidad

- **map(valor, minIn, maxIn, minOut, maxOut):** Re-mapea proporcionalmente un número de un rango a otro. Muy útil para convertir lecturas de sensores (0-1023) a formatos utilizables como PWM (0-255).
  - Ejemplo: int brillo = map(valorPot, 0, 1023, 0, 255);
- **constrain(valor, min, max):** Restringe un número para que se mantenga dentro de un rango determinado.

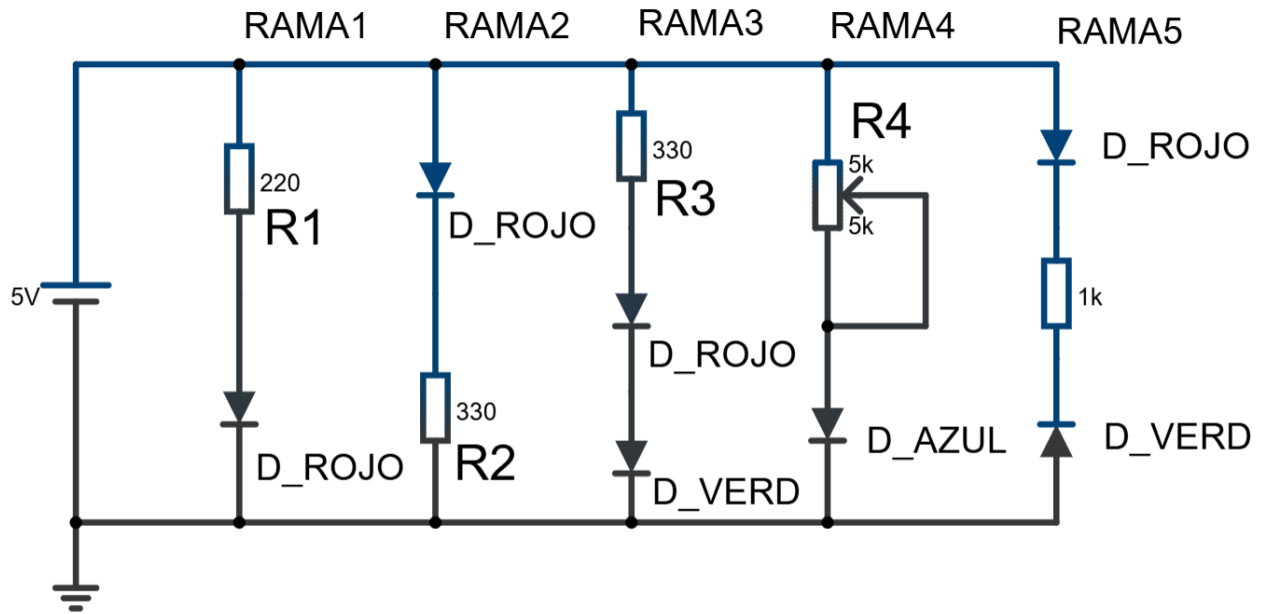
**Revise Los anexos para comprender  
más acerca del protoboard y su conexionado**

#### 4. Implementación en Protoboard

Analizar el efecto del valor de resistencia limitadora en la corriente y el brillo de un LED cuando se alimenta desde el riel 5V del Arduino (sin usar pines digitales).

**¡Recuerde!**

- No conectar LEDs sin resistencia.
- Anodo del LED al +5 V; cátodo a GND a través de la resistencia en serie (el orden serie no altera).



*Se pide.*

1. Montar el Circuito Mostrado en el Protoboard de acuerdo a cómo se piden las ramas.
2. **RAMAS 1 y 2:** Comparar el brillo cualitativamente (DIODO ROJO1 VS. DIODO ROJO2).
3. **RAMA 3:** ¿Qué sucede con el brillo de los diodos LED? ¿Por qué sucede esto?
4. **RAMA 4:** Configure el potenciómetro a la mitad y conecte el diodo led AZUL. Variando la perilla obtenga el brillo mínimo y máximo. En ambos casos, mida el nivel de tensión en el potenciómetro utilizando el multímetro como voltímetro.
5. **RAMA 5:** ¿Qué sucede con los diodos? ¿Por qué?

**Nota.** Para su informe grabe videos del funcionamiento y adjuntarlos en un drive.

### Checkpoint 4

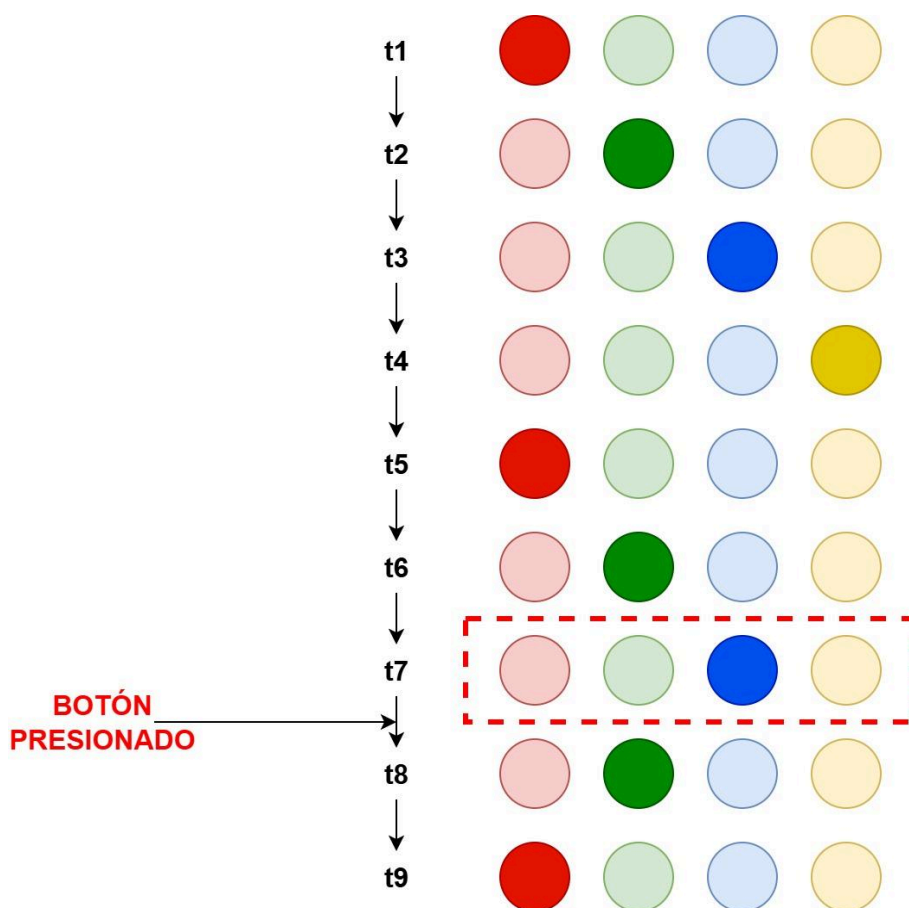
## 5. Control de Tiempos y Eventos Asíncronos

En base a la introducción a Arduino que usted ha recibido, implementará un sistema que integre el control dinámico de retardos y la lectura de eventos asíncronos (botón) para evitar el uso de código bloqueante.

### 5.1. Conexión.

- LEDs: Emplear cuatro diodos LED (Rojo, Azul, Verde, Amarillo). Conectar el ánodo de cada uno a pines digitales a través de sus respectivas resistencias (entre 100 Ohm y 330 Ohm); cátodo a GND.
- Botón: Un switch o pulsador conectado a un pin digital (se sugiere el pin 2 o 3).

**Nota.** Se debe utilizar configuración de botón externa en modo Pull-up o Pull-down.



### 5.2. Instrucciones.

- Los LEDs se encenderán secuencialmente en el siguiente orden: Rojo, Azul, Verde, Amarillo. Al finalizar, volverá al inicio (Rojo) y repetirá la secuencia de manera continua.
- El retardo inicial entre encendidos será de 2000 ms.
- Aceleración exponencial: En cada iteración (cambio de LED), el tiempo de retardo debe reducirse multiplicándose por 0.95 respecto al tiempo anterior ( $delay_{n+1} = 0.95 \times delay_n$ ).

- Al llegar a un retardo  $\leq 100$  ms, el sistema reinicia el tiempo a 1000 ms y repite.
- Entrada de reversa: Al presionar el botón, la secuencia cambia en el sentido inverso.
- Nota Crítica: Al presionar el botón el cambio debe realizarse instantáneamente, sin acabar el ciclo de espera (Por ejemplo, si se presiona el botón durante el retardo del LED Azul, el siguiente LED a encender debe ser el Rojo de inmediato). Solo se aceptará el checkpoint si se cumple esta condición.

### **Recomendaciones.**

- Declarar los pines de los LEDs como OUTPUT y el del botón como INPUT o INPUT\_PULLUP.
- Usar una variable para el retardo actual (p. ej., tDelay\_ms).
- Al detectar  $tDelay\_ms \leq 100$ , restablecer a 1000.
- Evitar valores  $< 0$  ms.
- Para lograr el cambio instantáneo requerido, el uso de la función delay() será insuficiente (código bloqueante). Se recomienda investigar e implementar el uso de la función millis() para hacer una lectura continua del tiempo, o bien interrupciones por hardware (attachInterrupt).
- Documentar y justificar en su informe el perfil temporal elegido y cómo solucionó el problema del retardo para que la respuesta sea inmediata.

### **Se pide**

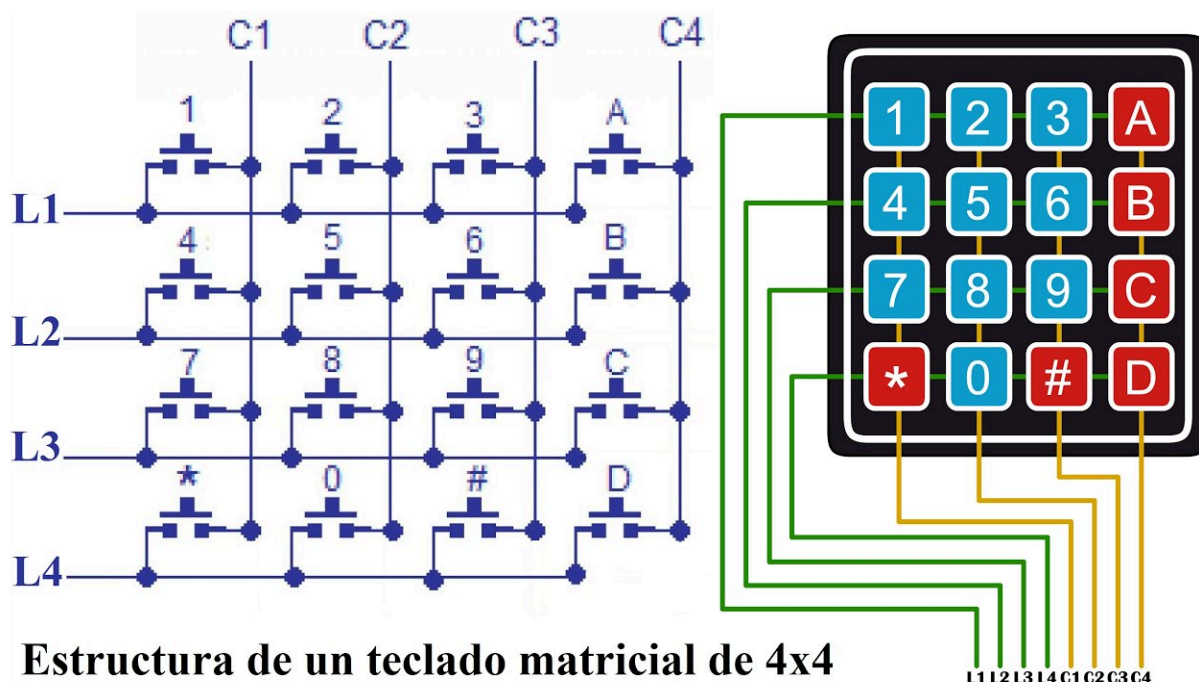
1. Realice el planteamiento (diagrama de flujo).
2. Realice la implementación en protoboard y demuestre el funcionamiento.
3. Mostrar y explicar al docente el funcionamiento del circuito, haciendo énfasis en cómo resolvió la lectura del botón.
4. Para su informe: Adjuntar así mismo una simulación.

**Nota. Para su informe grabe videos del funcionamiento y adjuntarlos en un drive.**

## **Checkpoint 5**

## 6. Sistema de Acceso Matricial y Feedback Serial

En esta etapa, se integrará un periférico de entrada múltiple (Teclado de membrana 4x4) y se utilizará el Monitor Serial para la depuración y visualización de datos. Se mantendrá la exigencia de usar código no bloqueante para gestionar eventos paralelos.



### 6.1. Conexión.

- Teclado 4x4: Conectar los 8 pines del teclado (4 filas y 4 columnas) a 8 pines digitales del Arduino (se sugiere del 4 al 11).
- LEDs: Emplear dos diodos LED (Verde y Rojo) conectados a pines digitales independientes a través de sus respectivas resistencias limitadoras; cátodo a GND.

### 6.2. Instrucciones.

El sistema funcionará como una cerradura electrónica básica supervisada mediante la consola serial.

- Al encender, el sistema debe inicializar el Monitor Serial a 9600 baudios e imprimir el mensaje: "Sistema Listo. Ingrese PIN de 4 dígitos:".
- Cada vez que se presione una tecla, el carácter correspondiente debe imprimirse en el Monitor Serial.
- Al presionar la tecla '#' (Enter), el sistema evaluará los últimos 4 caracteres ingresados contra una contraseña predefinida en el código (ej. "1234").
  - Si el PIN es correcto: Imprimir en el Monitor Serial "Acceso Concedido". El LED Verde se encenderá durante 3000 ms y luego se apagará.



- Si el PIN está incorrecto: Imprimir en el Monitor Serial "Error de validación". El LED Rojo parpadea rápidamente (intervalos de 250 ms) durante un lapso total de 2000 ms.
- Nota Crítica de Tiempos: El encendido del LED Verde o el parpadeo del LED Rojo deben realizarse utilizando la función millis(). Durante el tiempo que los LEDs estén activos, el microcontrolador debe seguir escaneando el teclado. Si el usuario presiona una tecla mientras el LED Verde está encendido o el Rojo está parpadeando, el **Monitor Serial** debe registrar instantáneamente: "Sistema ocupado, espere...", sin alterar ni interrumpir el ciclo de tiempo de los LEDs.

### Recomendaciones.

- Puede hacer uso de la librería Keypad.h> para facilitar la lectura de la matriz, o bien implementar el barrido digital (escaneo de filas y columnas) de manera manual.
- Utilice variables de estado (ej. bool sistema Ocupado) para bloquear la acumulación de caracteres nuevos mientras los LEDs están ejecutando su rutina de tiempo.
- Recuerde limpiar el buffer o la variable de almacenamiento de su contraseña después de cada evaluación (al presionar '#').

### Se pide:

1. Realice el planteamiento del algoritmo (diagrama de flujo o máquina de estados) detallando cómo evitará bloquear la lectura del teclado.
2. Realice la implementación en protoboard y demuestre el funcionamiento.
3. Mostrar y explicar al docente el funcionamiento del circuito, demostrando físicamente qué sucede en el Monitor Serial si se presionan teclas durante el tiempo de validación (LEDs encendidos/parpadeando).
4. Para su informe: Adjuntar capturas de pantalla del Monitor Serial evidenciando los casos de éxito, error y de "sistema ocupado", además de la simulación correspondiente.

Nota. Para su informe grabe videos del funcionamiento y adjuntarlos en un drive.

## Checkpoint 6

## 7. Checkpoint Opcional: Visualización Dinámica en Matriz 8x8 (MAX7219)

En esta experiencia, se introducirá el control de un periférico de salida avanzado utilizando el protocolo de comunicación SPI. Para mantener la simplicidad lógica, el objetivo será diseñar dos símbolos o íconos estáticos y utilizar una interrupción o lectura no bloqueante de un botón para alternar entre ellos.

**Nota.** Puede referirse al Anexo K para mayor información del componente.

### 7.1. Conexión.

- Matriz 8x8 (Módulo MAX7219):
  - VCC al pin de 5V del Arduino.
  - GND al pin GND.
  - DIN (Data In) a un pin digital (ej. Pin 11).
  - CS (Chip Select / LOAD) a un pin digital (ej. Pin 10).
  - CLK (Clock) a un pin digital (ej. Pin 13).
- Botón: Conectar un botón a un pin digital (ej. Pin 2) configurado en modo INPUT\_PULLUP.

### 7.2. Instrucciones.

1. Diseñe dos íconos de 8x8 píxeles (por ejemplo: una "Cara Feliz" y una "Cara Triste", o un "Check" y una "X").
2. Traduzca estos diseños a arreglos (arrays) de bytes en su código (revise el ANEXO K para más detalles).
3. El sistema debe inicializar mostrando el "Ícono 1" por defecto.
4. Al detectar la pulsación del botón (evento asíncrono), la matriz debe limpiar la pantalla y mostrar inmediatamente el "Ícono 2". Al soltar o volver a presionar (según su lógica de programación), debe regresar al "Ícono 1".
5. Opcional (Puntos extra): Incorpore el potenciómetro a un pin analógico (ej. A0) y utilice la función de la librería para controlar el nivel de brillo de la matriz en tiempo real, mapeando el valor del ADC (0-1023) al rango de intensidad de la matriz (0-15).

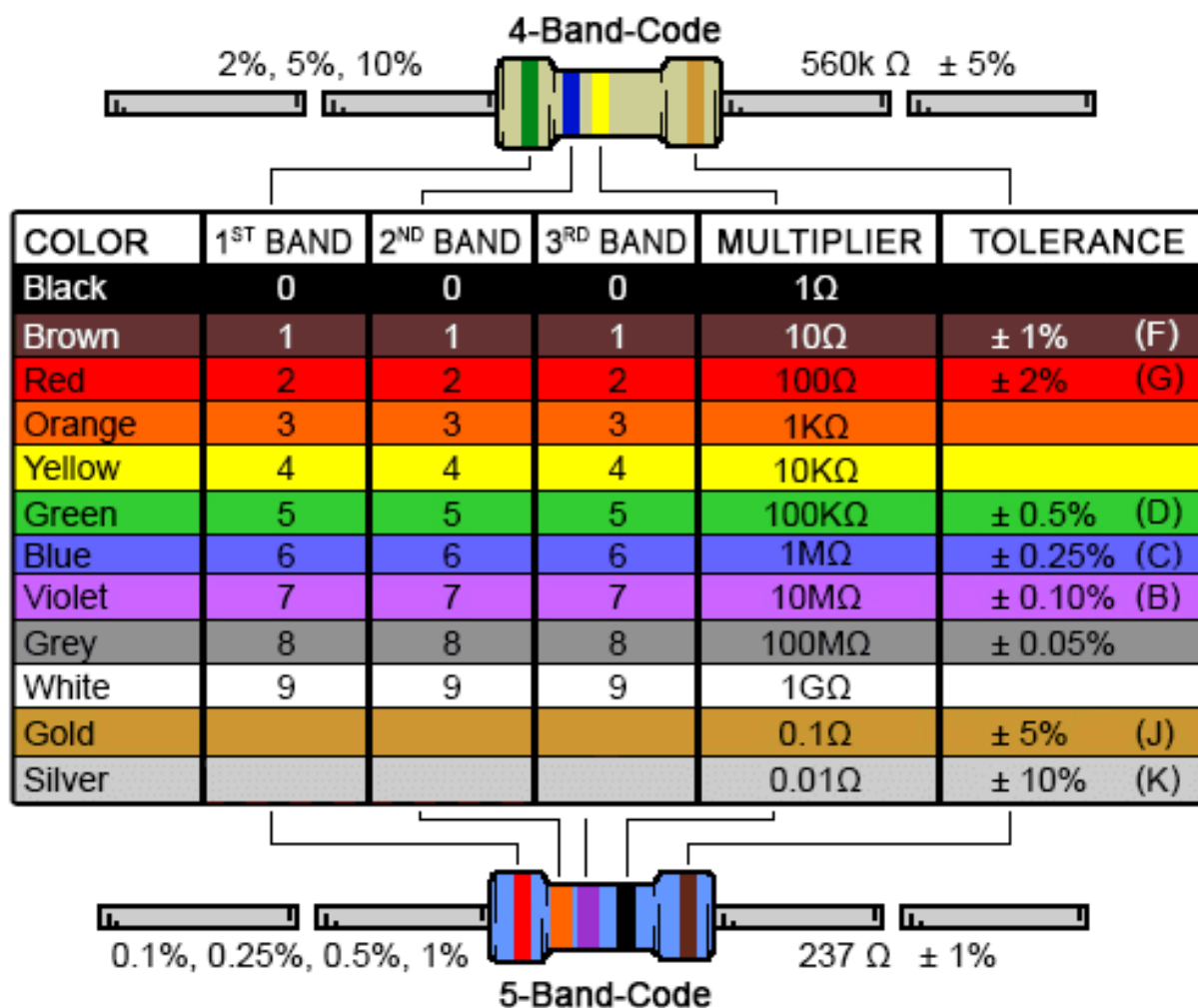
### Se pide:

- Implementar el circuito en el protoboard.
- Mostrar y explicar al docente el mapa de bits (los valores en hexadecimal o binario) que generan sus dos íconos.
- Demostrar el cambio inmediato de imagen al accionar el botón.
- Para su informe: Incluir el código fuente comentado y capturas de los íconos diseñados. Grabe un video del funcionamiento y adjúntelo en el drive.

## Checkpoint 7

## ANEXOS

### ANEXO A. Códigos de Colores de las Resistencias.

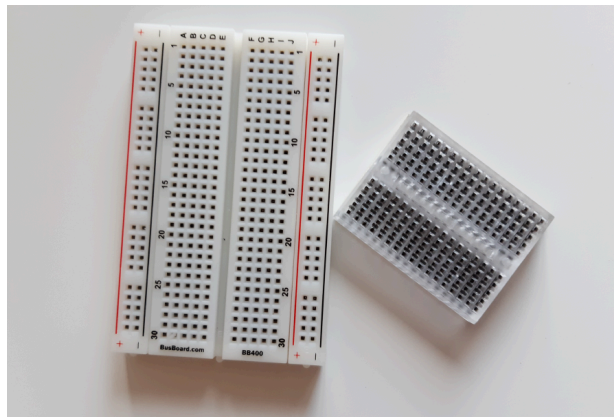


Recordar que para conectar **diodes LED** utilizar resistencias entre **100 Ohm y 330 Ohm**.

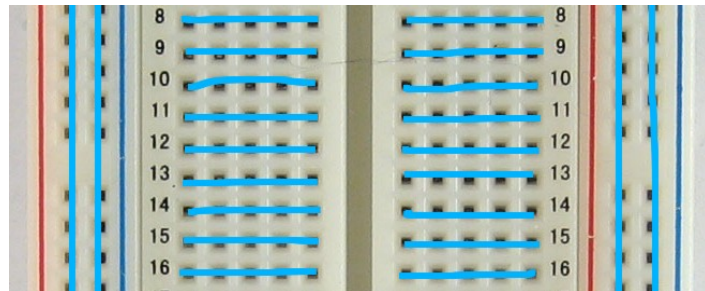
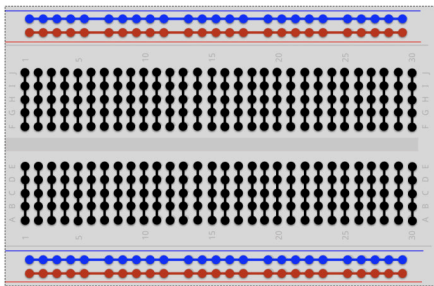
Así como para utilizar la configuración de botones pull-up y pull-down, usar resistencias de 4.7k Ohm en adelante.

### ANEXO B. Uso del Protoboard

Un **protoboard**, también conocido como **breadboard**, es una herramienta utilizada para armar circuitos electrónicos de forma temporal sin necesidad de soldar los componentes. Está compuesto por una base plástica con una matriz de agujeros conectados internamente por tiras metálicas. Los componentes, como resistencias, transistores, LEDs, cables, entre otros, se insertan en los agujeros para crear conexiones eléctricas.

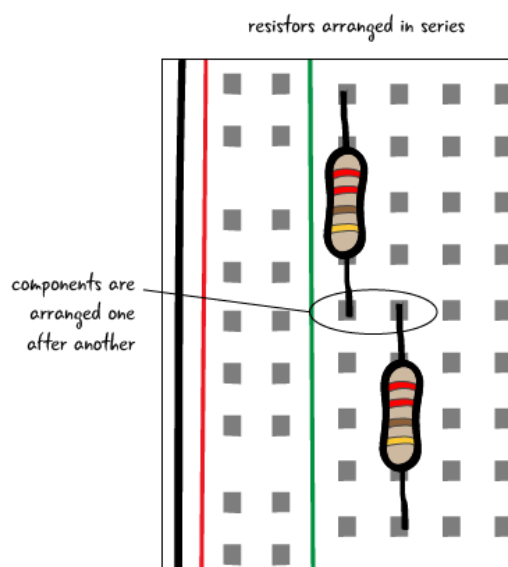


La conexiones del protoboard se dan de la manera siguiente: (1) Los orificios laterales están conectados en serie, mientras que los centrales se conectan en serie por cada columna, pero de columna a columna se encuentran en paralelo.



## 6.1. Conexionado básico

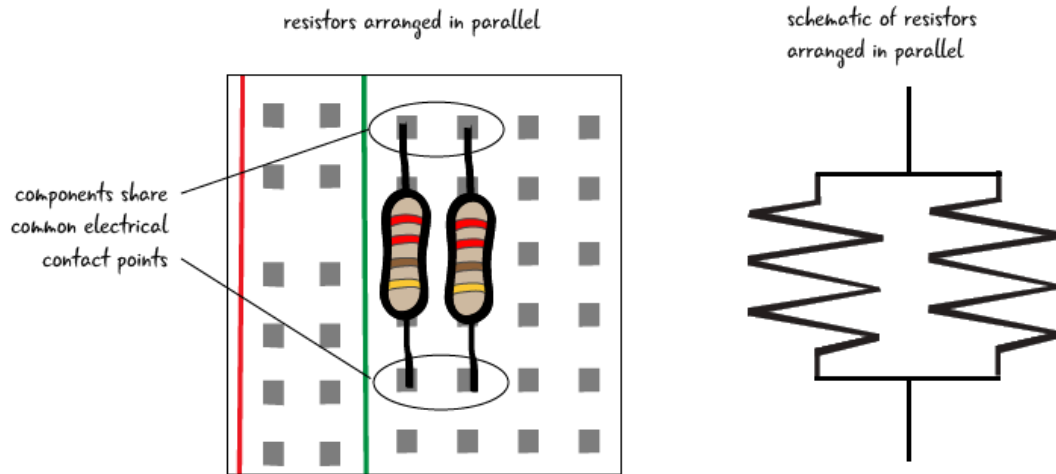
### 6.1.1. Conexión en serie



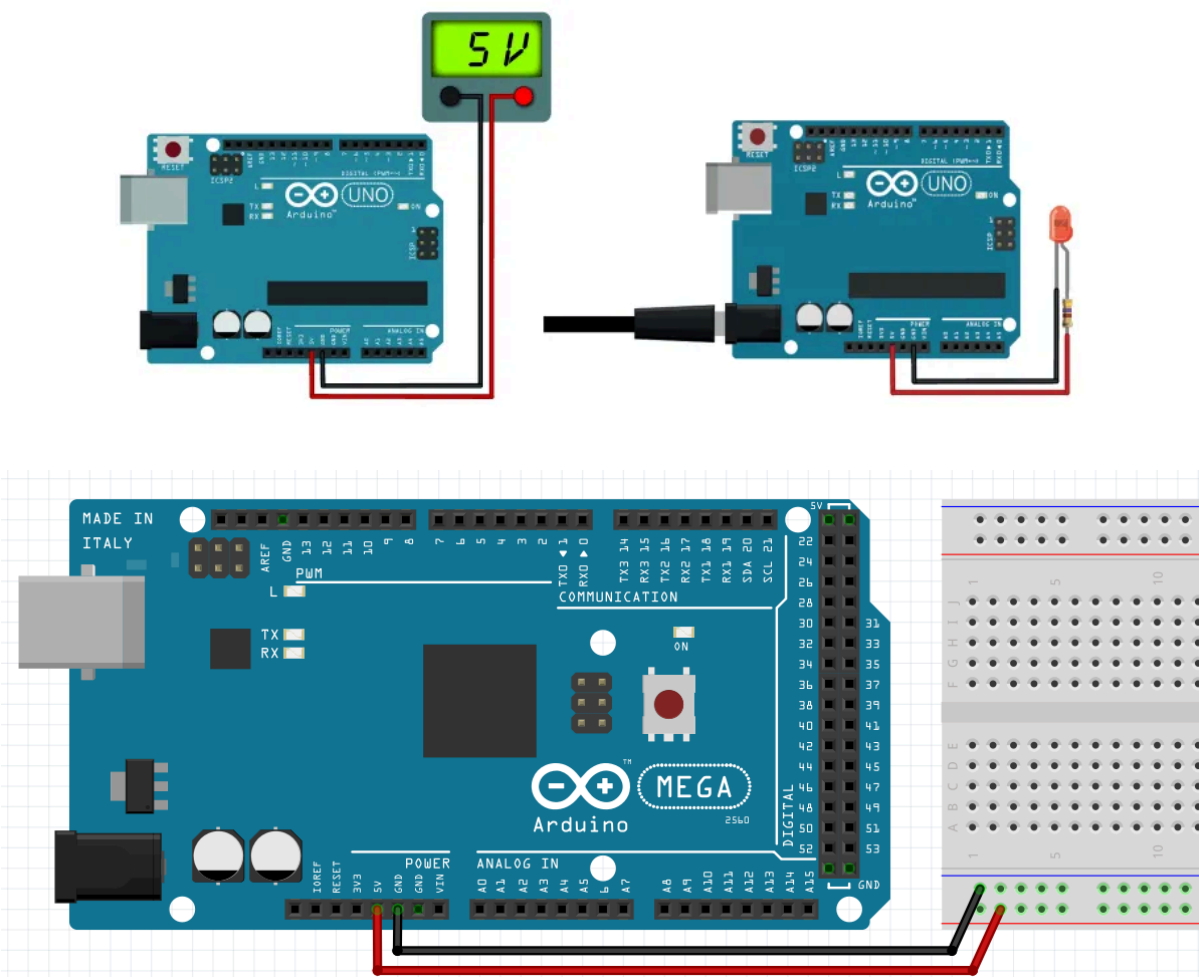
schematic of resistors arranged in series



### 6.1.2. Conexión en paralelo

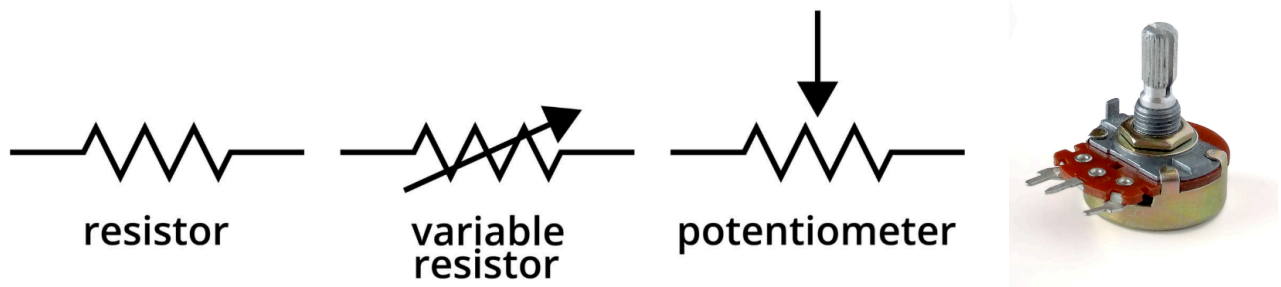


## ANEXO C. Alimentación de protoboard con Arduino



## ANEXO D. Potenciómetro

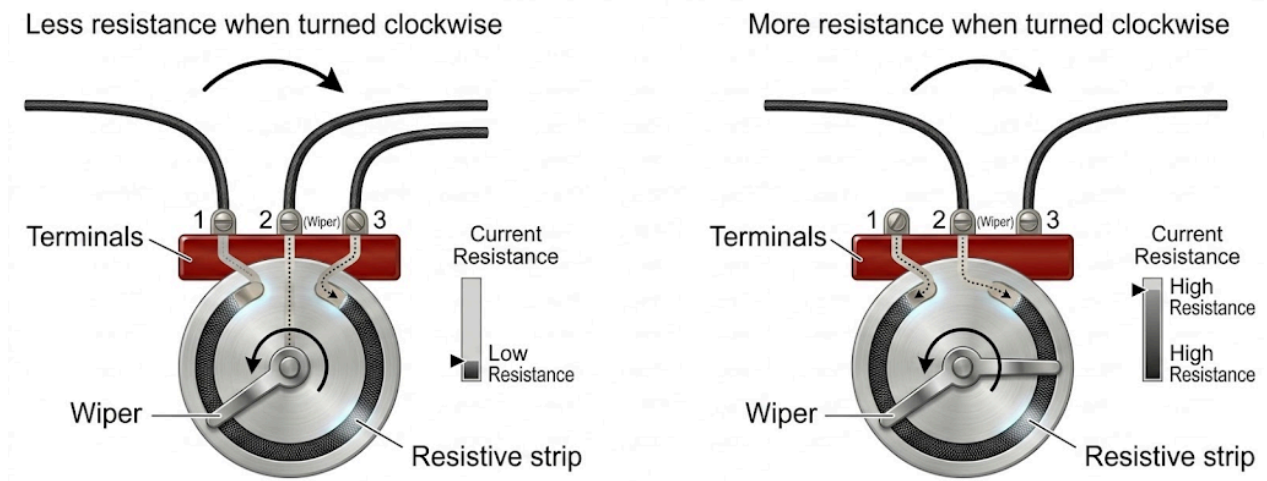
Un potenciómetro es esencialmente un resistor variable de tres terminales. Permite ajustar manualmente el valor de la resistencia, lo que lo hace ideal para controlar variables como el voltaje o la corriente en una parte específica de un circuito (por ejemplo, para ajustar referencias en simulaciones o variar la frecuencia de un reloj digital). Está compuesto por una pista resistiva (generalmente de carbón) conectada a los dos terminales extremos. Un tercer terminal (el central, o *wiper*) está conectado a un contacto móvil que se desliza sobre la pista cuando se gira la perilla.



Se usa en dos tipos de conexionado dentro de los circuitos electrónicos.

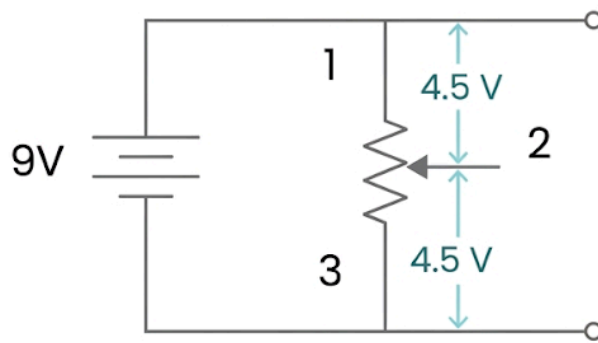
### 8.1. Configuración como resistencia variable (reóstato)

Si solo se utiliza un terminal extremo y el terminal central, el dispositivo se comporta como una resistencia que varía de cero a su valor nominal máximo.



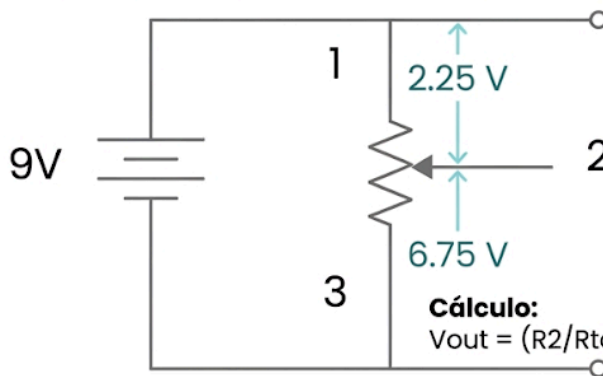
### 8.2. Configuración como divisor de tensión

Si se aplica un voltaje entre los dos terminales extremos, el terminal central entregará un voltaje proporcional a su posición.



**Potenciómetro  
en el punto  
medio.**

2: CASO DE 3/4 DE POSICIÓN



**Perilla a 3/4 de su posición.**

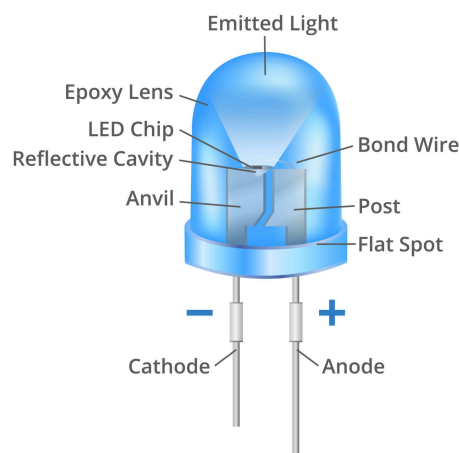
$$9V/4=2.25V$$

**Cálculo:**  
 $V_{out} = (R_2/R_{total}) * V_{in}$

## ANEXO E. El Diodo LED (Light Emitting Diode)

Un LED es un semiconductor que emite luz cuando la corriente eléctrica lo atraviesa en un sentido específico. A diferencia de las bombillas tradicionales (o de un resistor), el LED es un componente polarizado, lo que significa que la dirección en la que se conecta es crítica para que funcione (y para evitar que se queme).

### Light Emitting Diode



- **Ánodo y Cátodo:** El terminal positivo se llama ánodo (usualmente la pata más larga) y el negativo se llama cátodo (la pata más corta, que suele coincidir con un lado plano en el encapsulado plástico del LED).



- Voltaje directo (Forward Voltage): Todo LED requiere un voltaje mínimo para encenderse (típicamente entre 1.8V y 3.3V, dependiendo del color).
- Regla de oro: Un LED siempre debe ir acompañado de un resistor limitador de corriente en serie. Si se conecta directamente a una fuente de alimentación mayor a su voltaje directo sin protección, absorberá demasiada corriente y se destruirá casi instantáneamente.

## ANEXO F. Instrumentos y Formas de Medición

Para garantizar el correcto funcionamiento de los circuitos y la seguridad durante las sesiones prácticas, es indispensable dominar las técnicas de medición. A continuación, se detallan los procedimientos estándar para medir corriente y tensión.

### 10.1. Medición de Intensidad de Corriente ("I")

La intensidad de corriente indica el flujo de electrones a través de un conductor y se mide utilizando un amperímetro (ya sea en su versión analógica o como función dentro de un multímetro digital).

- Unidad de medida: Amperio [A].
- Método de conexión: El instrumento debe conectarse estrictamente en SERIE con la carga.
- Procedimiento: Para lograr esto, se debe "abrir" el circuito sobre la línea o fase que se desea medir e intercalar las puntas del amperímetro. Esto obliga a que toda la corriente pase a través del instrumento de medición.



### 10.2. Medición de Tensión o Voltaje ("V" o "U")

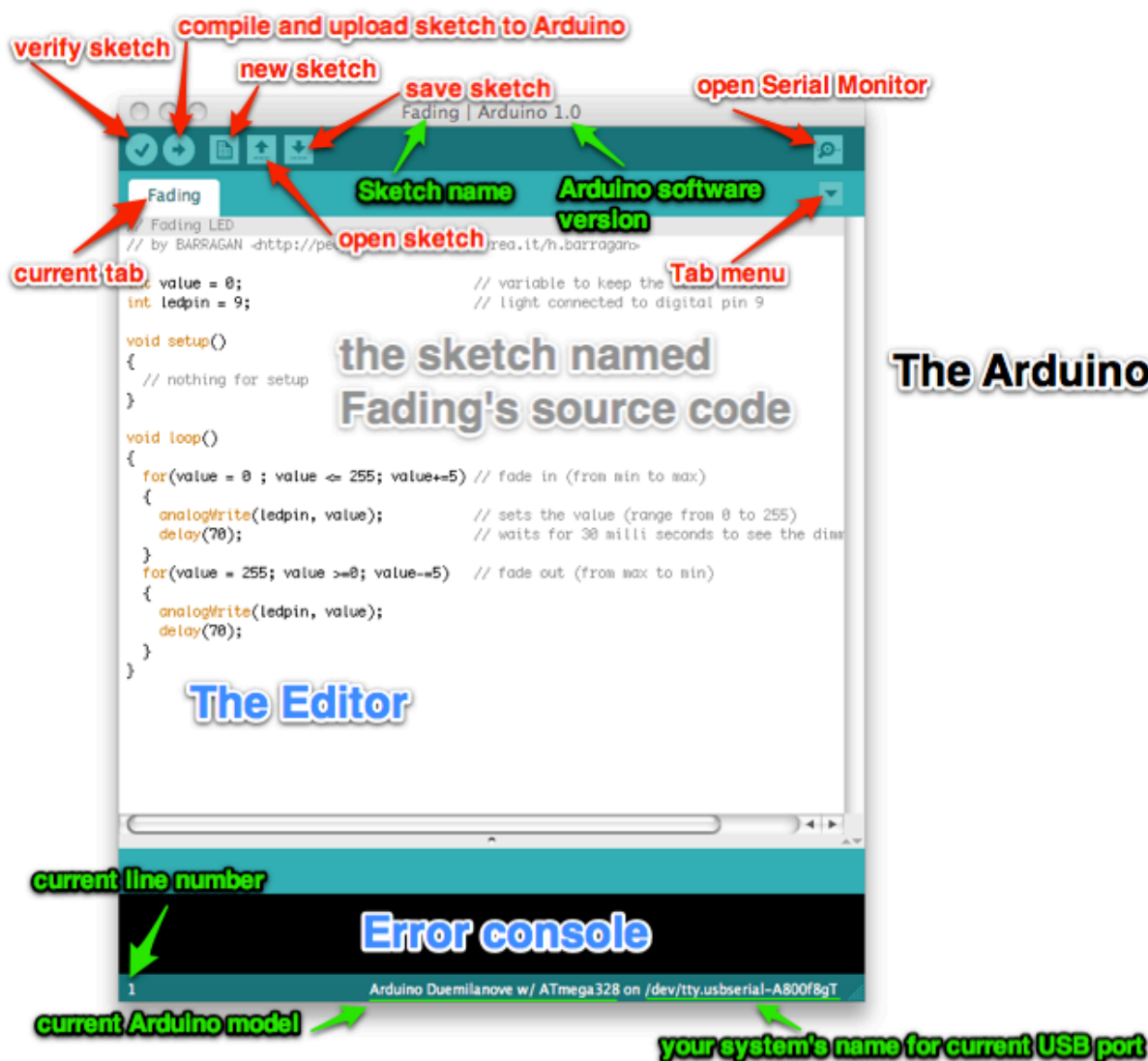
La tensión representa la diferencia de potencial eléctrico entre dos puntos y se mide utilizando un voltímetro.

- Unidad de medida: Voltio [V].
- Método de conexión: El instrumento debe conectarse de forma PARALELA a la carga o a la fuente de alimentación (tomacorriente, barra, batería, etc.). No es necesario abrir el circuito.
- Procedimiento en redes eléctricas:
  - Circuito monofásico (220V): Se coloca una punta del instrumento sobre la línea de fase y la punta restante sobre la línea de neutro.
  - Circuito trifásico (380V): Las puntas del instrumento se deben colocar sobre cada una de las fases (para medir tensión entre fases), o entre cualquiera de las tres fases y el neutro.

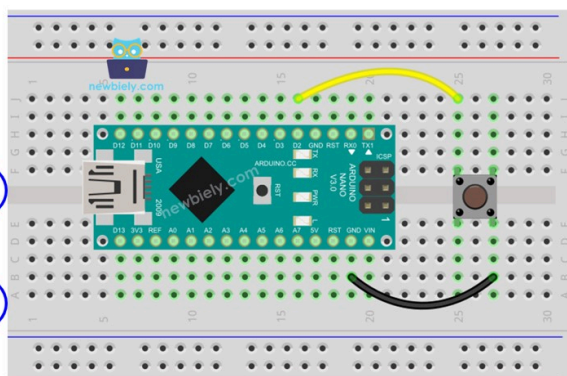
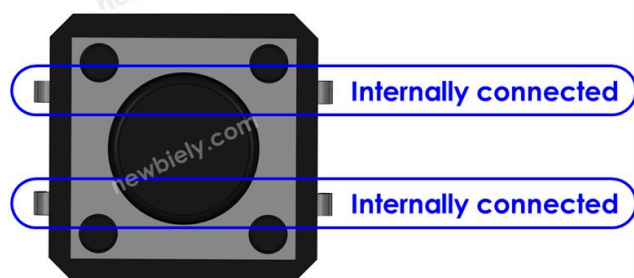
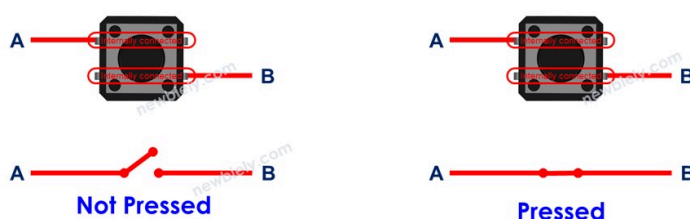
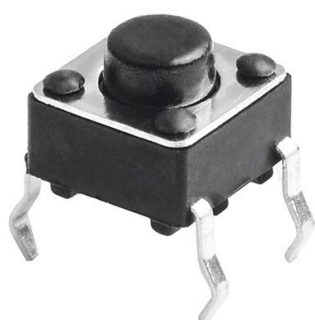
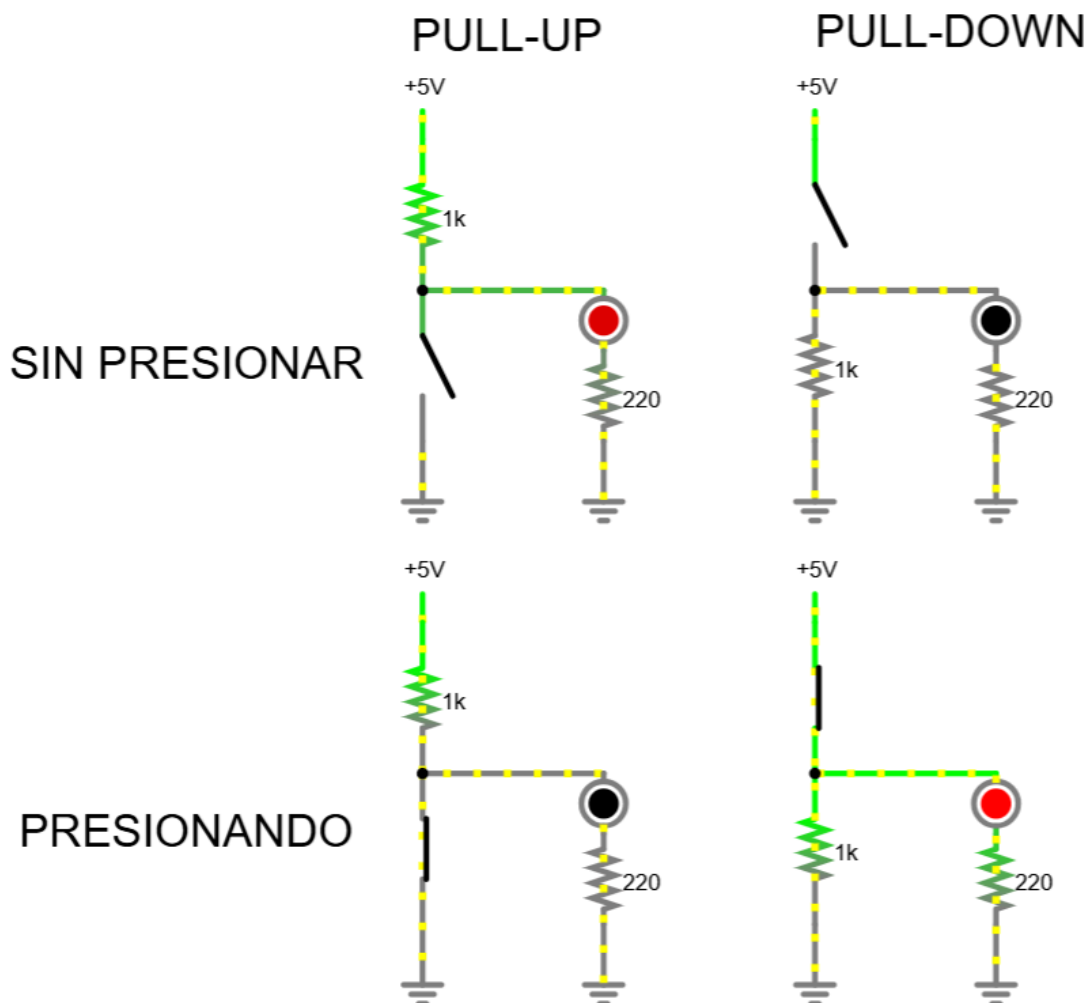


|                                       |                                                                                                                          |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Unidad de medición: <b>VOLTIO [V]</b> | <br>Símbolo General del<br>Voltímetro |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------|

## ANEXO G. Programación con Arduino: Interfaz de Arduino



## ANEXO H. Botón con Arduino.



## ANEXO I. Monitor Serial.

Usar referencia:

<https://aprendiendoarduino.wordpress.com/category/monitor-serie/>

### 1. Definición y Utilidad

El Monitor Serial es una herramienta integrada en el entorno de desarrollo (IDE) de Arduino que permite la comunicación bidireccional entre la computadora y el microcontrolador mediante el protocolo UART (a través del cable USB).

Esta interfaz es fundamental para la depuración de código (debugging), ya que permite al programador visualizar el estado de las variables, los resultados de los sensores en tiempo real y los mensajes de error. Así mismo, permite controlar el sistema enviando caracteres o cadenas de texto desde la computadora hacia la placa.



### 2. Principio de Funcionamiento (UART) y Conexión

La comunicación serial en Arduino se basa en hardware UART (Universal Asynchronous Receiver-Transmitter). Al ser asíncrona, no existe una señal de reloj (clock) compartida que sincronice a ambos dispositivos. En su lugar, los dispositivos se conectan cruzando sus líneas de transmisión y recepción:

- TX (Transmit): Pin por donde el dispositivo envía los datos.
- RX (Receive): Pin por donde el dispositivo recibe los datos.
- GND (Ground): Ambos dispositivos deben compartir la misma referencia de tierra para que los niveles de voltaje (0V y 5V) sean interpretados correctamente.

**Nota:** Cuando conectas el Arduino por USB a la computadora, un chip conversor en la placa realiza internamente esta conexión hacia los pines digitales 0 (RX) y 1 (TX).



### 3. El Baudrate (Velocidad de Transmisión)

Como no hay una señal de reloj que indique cuándo empieza y termina un bit, ambos dispositivos deben acordar previamente a qué velocidad van a "hablar" y "escuchar". Esta velocidad se mide en Baudios (Baud rate), que en este contexto equivale a bits por segundo (bps).

- Si configuras el Arduino a 9600 baudios, significa que enviará y leerá 9600 ceros o unos por segundo.
- La computadora (el Monitor Serial) debe estar configurada a la misma velocidad. Si el Arduino envía datos a 9600 baudios pero el Monitor Serial está configurado para "escuchar" a 115200 baudios, la computadora leerá las señales a destiempo, traduciendo los pulsos eléctricos como "basura" (caracteres extraños como ???~).

### 4. Funciones Principales

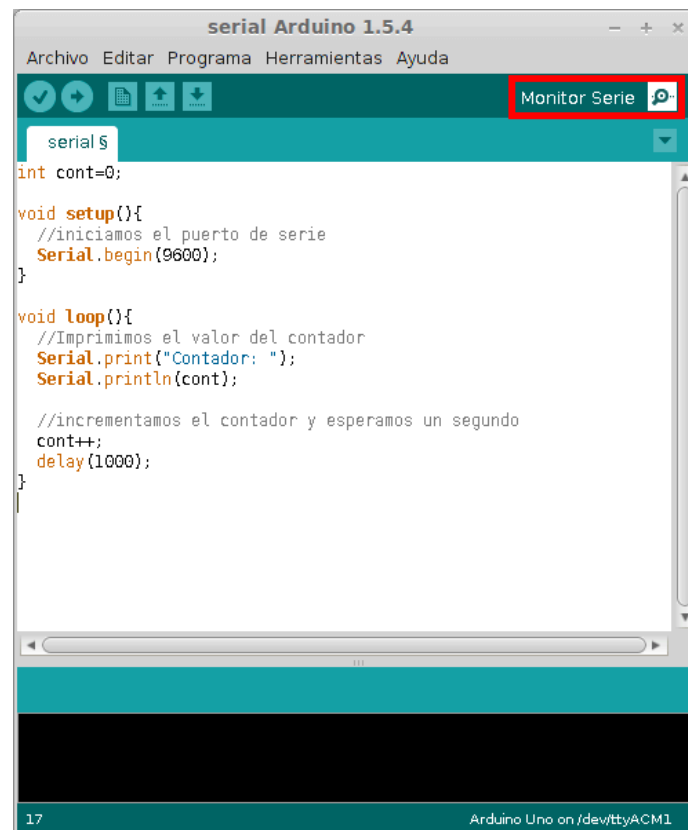
Para utilizar el puerto serial en Arduino, se emplean las siguientes funciones básicas dentro del código:

- `Serial.begin(velocidad)`: Inicializa la comunicación serial por hardware. Debe colocarse dentro del `setup()`. Los parámetros más comunes son 9600 o 115200 baudios.
- `Serial.print(dato)` y `Serial.println(dato)`: Envía datos al monitor serial. La instrucción `println` añade un salto de línea al final del mensaje.
- `Serial.available()`: Retorna el número de bytes (caracteres) disponibles para ser leídos en el buffer de recepción. Se usa típicamente en un condicional `if (Serial.available() > 0)` para asegurar que haya datos antes de leer.
- `Serial.read()`: Lee el primer byte de los datos entrantes disponibles en el buffer serial.

### 5. Apertura y Configuración

Para abrir el Monitor Serial en el IDE de Arduino:

1. Haga clic en el icono de la lupa ubicado en la esquina superior derecha del IDE, o presione `Ctrl + Shift + M`.
2. Asegúrese de que el menú desplegable de baudios (esquina inferior derecha de la ventana del monitor) coincida exactamente con el valor declarado en su `Serial.begin()`.



## 6. Ejemplo Práctico: Control de LED vía Consola

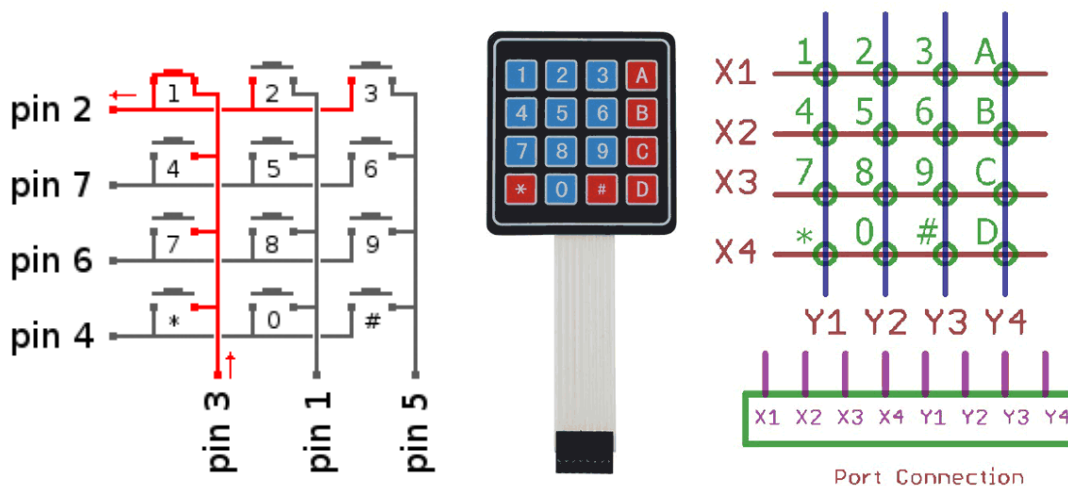
El siguiente código demuestra cómo encender y apagar el LED integrado en la placa Arduino enviando caracteres específicos mediante el Monitor Serial.

```
1  const int ledPin = 13; // Pin del LED integrado en la placa
2  char comando;          // Variable para almacenar el carácter recibido
3
4  void setup() {
5      // 1. Inicializar la comunicación serial a 9600 baudios
6      Serial.begin(9600);
7      pinMode(ledPin, OUTPUT);
8
9      // Imprimir un mensaje inicial
10     Serial.println("Sistema Listo.");
11     Serial.println("Envíe 'b' para encender el LED o 'a' para apagarlo.");
12 }
13
14 void loop() {
15     // 2. Verificar si hay datos ingresados por el usuario esperando en el buffer
16     if (Serial.available() > 0) {
17
18         // 3. Leer el dato entrante
19         comando = Serial.read();
20
21         // 4. Evaluar el dato recibido y ejecutar la acción
22         if (comando == 'b') {
23             digitalWrite(ledPin, HIGH);
24             Serial.println("Estado: LED Encendido");
25         }
26         else if (comando == 'a') {
27             digitalWrite(ledPin, LOW);
28             Serial.println("Estado: LED Apagado");
29         }
30     }
31 }
```

## ANEXO J. Teclado Matricial

Usar referencia:

<https://controlautomaticoeducacion.com/sistemas-embebidos/arduino/teclado-matricial-keypad/>



Una forma rápida de usar un Teclado con Arduino, es valernos de su librería Keypad (<https://www.arduino.cc/reference/en/libraries/keypad/>), bastante sencilla de entender, para el caso de un teclado matricial 4×4 (KEYPAD 4×4) tenemos:

```
#include <Keypad.h>
const byte filas = 4;
const byte columnas = 4;
byte pinesFilas[] = {9,8,7,6};
byte pinesColumnas[] = {5,4,3,2};
char teclas[4][4] = {{ '1','2','3','A'},
                     { '4','5','6','B'},
                     { '7','8','9','C'},
                     { '*', '0', '#', 'D' }};

Keypad teclado1 = Keypad( makeKeymap(teclas), pinesFilas, pinesColumnas, filas, columnas);
void setup() {
  Serial.begin(9600);
  Serial.println("Teclado 4x4 con Biblioteca Keypad");
  Serial.println();
}
void loop() {
  //Verifica si alguna tecla fue presionada
  char tecla_presionada = teclado1.getKey();

  //Monitor Serial
  if (tecla_presionada)
  {
    Serial.print("Tecla: ");
    Serial.println(tecla_presionada);
  }
}
```

## ANEXO K. Matriz de Puntos 8x8 con Controlador MAX7219

### 1. Definición y Principio de Funcionamiento

Una matriz de LED 8x8 contiene 64 diodos individuales organizados en filas y columnas. Para evitar usar 64 pines del microcontrolador, se utiliza una técnica de barrido visual (multiplexado). El módulo comercial mostrado en el archivo `image_d09c08.jpg` simplifica enormemente este proceso al incorporar el circuito integrado MAX7219.

Este chip asume la carga de trabajo del barrido a alta frecuencia (aprovechando la persistencia retiniana del ojo humano) y se comunica con el Arduino mediante un protocolo serie síncrono muy rápido llamado SPI (Serial Peripheral Interface), requiriendo únicamente 3 pines de datos.

### 2. Conexiones (Pines del Módulo)

El módulo cuenta con 5 pines de entrada (IN) en un extremo y, a menudo, 5 pines de salida (OUT) en el otro, lo que permite conectar múltiples matrices en cascada.

- VCC: Alimentación (5V).
- GND: Tierra.
- DIN (Data In): Recibe los datos enviados por el Arduino.
- CS / LOAD (Chip Select): Habilita el módulo para recibir y procesar los datos.
- CLK (Clock): Señal de reloj generada por el Arduino para sincronizar la transmisión de cada bit.

### 3. Representación de Imágenes (Mapeo de Bits)

Cada imagen en la matriz se concibe como un conjunto de 8 filas, donde cada fila tiene 8 LEDs. En programación, esto equivale a 1 byte por fila (donde 1 es LED encendido y 0 es apagado). Una imagen completa requiere un Array de 8 bytes.

Ejemplo de diseño de un "Cuadrado":

```
1  byte cuadrado[8] = {
2      B11111111, // Fila 0
3      B10000001, // Fila 1
4      B10000001, // Fila 2
5      B10000001, // Fila 3
6      B10000001, // Fila 4
7      B10000001, // Fila 5
8      B10000001, // Fila 6
9      B11111111  // Fila 7
10 };
11 // Nota: También se pueden expresar en formato
12 // hexadecimal (ej. 0xFF en vez de B11111111).
```

4. Librería Recomendada y Código Base Para abstraer la complejidad del protocolo SPI, se recomienda instalar y usar la librería <LedControl.h> desde el gestor de librerías del IDE de Arduino.



Funciones principales de LedControl:

- `LedControl lc = LedControl(DIN, CLK, CS, Num_Matrices);` : Inicializa el objeto.
- `lc.shutdown(0, false);` : Despierta la matriz (por defecto inicia en modo ahorro).
- `lc.setIntensity(0, brillo);` : Ajusta el brillo (de 0 a 15).
- `lc.clearDisplay(0);` : Apaga todos los LEDs.
- `lc.setRow(0, fila, valorByte);` : Dibuja un byte entero en una fila específica.

Ejemplo de inicialización:

```
1  #include <LedControl.h>
2
3  // Pines: DIN=11, CLK=13, CS=10. El '1' indica que solo hay 1 matriz conectada.
4  LedControl lc = LedControl(11, 13, 10, 1);
5
6  void setup() {
7      lc.shutdown(0, false);      // Enciende la matriz 0
8      lc.setIntensity(0, 8);      // Establece brillo medio (0 a 15)
9      lc.clearDisplay(0);         // Limpia la pantalla
10 }
```